

Classic Fortress

Trading-Bot System & Dashboard Whitepaper

KHF Zero-Cost Trading Bot - External technical review - Not investment advice

Classic Fortress (Trading-Bot) - System & Dashboard Whitepaper

Document purpose: External technical review of the Classic Fortress multi-agent trading platform (trading-bot repo), its Command Center dashboard, metrics, controls, and self-improvement loops.

Canonical filename: docs/TRADING_BOT_WHITEPAPER.md

PDF export: docs/TRADING_BOT_WHITEPAPER.pdf (generate via scripts/generate_whitepaper_pdf.py)

Product names: KHF Zero-Cost Trading Bot, Classic Fortress, Fortress

Version: Command Center (June 2026)

Primary reference: docs/TRADING_BOT_SYSTEM_BLUEPRINT.md

Disclaimer: Paper-first autonomous trading research infrastructure. Not investment advice.

1. Executive summary

Classic Fortress is a **rules-driven, multi-agent** US equities and options trading system on **Alpaca**, designed for **bounded automation** - not unconstrained AI execution. It is the **production stack** on the Oracle VM; the sibling **fortress-ai** repo runs a separate LLM-centric agent for A/B comparison.

Dimension Classic Fortress (this repo) Fortress AI (sibling) Architecture 60+ specialized agents, cron-driven Unified LLM agent + skim swarm Decision model Deterministic rules + optional LLM advisory DeepSeek observe-reason-act loop Dashboard port 8083 8050 Default book \$20,000 paper Separate Alpaca account Scheduling Cron (screen 2x/day, monitor 5 min) RTH loop every 5 min Primary P&L data/pnl_ledger.jsonl data/ai_decisions.jsonl + skim logs

Core promise: Hard risk/compliance gates (pre_trade_gate, risk_guardian, policy profiles) before any order reaches Alpaca. June 2026 adds **Classic SI** (integrity queue + fill-recency entry loosen + screener relax), **unified adaptive RSI** across screener prefilter and entry gate, and **chunked exits** under **FORTRESS_MAX_ORDER_NOTIONAL_USD**.

2. System architecture

2.1 High-level flow

Cron / systemd
-> orchestrator.py (CLI)
-> Intelligence (regime, sentiment, scouts, analysts, CIO)
-> Screener -> Entry -> Risk Guardian -> Pre-Trade Gate -> Alpaca
-> Exit Monitor (every 5 min RTH)
-> Post-close: Intelligence Brief, Recursive Evolution
-> dashboard/command_center.py (operator UI)

2.2 Core entrypoints

Component Path Role Orchestrator orchestrator.py Single CLI for all workflows Cron wrapper scripts/cron_run.sh venv + heartbeats + job dispatch Command Center dashboard/command_center.py Flask dashboard + 50+ REST APIs Health check_health.py Process/cron/data sanity

2.3 Config layers

Layer Path Contents Runtime config/fortress_runtime.yaml Agent toggles, portfolio default, LLM provider Policy config/policy_profiles.json Risk/screening/execution profiles Strategy allocation config/strategy_allocation.yaml Day/swing/position sleeves (30/40/30) Universe config/watchlist.json , international_universe.yaml Tickers Secrets .env Alpaca keys, halt, execution mode

2.4 Production schedule (typical)

Job Schedule Command Intraday screener Every 30 min 9-16 ET python3 -m agents.screener_agent (via

cron_run.sh screener) Full screen 14:35, 14:50, 15:05, 15:35 ET weekdays orchestrator.py screen Exit monitor Every 5 min 9-16 ET orchestrator.py monitor Regime detector 30 min RTH agents.regime_detector Sentiment velocity 20 min RTH agents.sentiment_velocity_agent Intelligence brief ~17:00 ET post-close generate_intelligence_brief Recursive evolution Post-close orchestrator.py evolve Ops autofix Every 15 min ops_autofix_agent

3. Trading pipeline (detailed)

3.1 Daily screening (orchestrator screen)

1. **Screener** (screener_agent) - RSI drop, volume, watchlist funnel; **adaptive RSI ceiling** from `utils/adaptive_rsi.py` raises tier-1 `rsi_threshold` when fill-recency SI is active (must match entry gate)
2. **Enrichment** - vision/news/fundamental paths (optional)
3. **Entry evaluation** (entry_agent) - per-candidate gates; uses `loosen_context()` + `adaptive_rsi_ceiling()`
4. **Risk Guardian** - circuit breaker, streak limits, VIX tiers
5. **Pre-trade gate** - compliance, halt, spread, notional caps
6. **Execution** - Alpaca submit or HITL queue (`pending_execution_queue.json`)

Artifacts: `daily_signals_YYYYMMDD.json`, `decisions_log.jsonl`, `trust_ledger.jsonl`

3.2 Exit monitoring (orchestrator monitor)

- **Exit monitor** - tiered exits, trailing stops, option expiry
- **Chunked sells** - `utils/order_sizer.py` splits exits exceeding `FORTRESS_MAX_ORDER_NOTIONAL_USD` via `plan_chunked_exit()` / `submit_chunked_sell_orders()`; wired in `orchestrator.py` `execute_sell_order()`
- **Output:** `exit_signals_YYYYMMDD.json`, updates to `pnl_ledger.jsonl`
- **Log marker:** `chunked_exit`

3.3 Classic SI pipeline (June 2026)

Parallel to recursive evolution - runs on integrity scan + evolve cron:

Module Role `utils/integrity_diagnostics.py` Detect classic fill recency, zero-candidate streaks, `adaptive_rsi_screener_drift`, sibling fortress-ai findings `utils/si_recommendation_queue.py` Same schema as fortress-ai sibling queue `utils/classic_si_autonomous.py` Heuristic assess + bounded auto-apply `utils/fill_recency_entry.py` Entry-aware fill recency; bounded RSI/LLM relax when no recent fill `utils/classic_si_entry.py` Auto-relax entry gate on persistent fill-recency gap `utils/classic_si_screener.py` Bear/ranging tier-1 relax on zero-candidate streaks `utils/adaptive_rsi.py` Single adaptive RSI ceiling - merges current params, fill-recency, entry SI overrides `utils/classic_screening_hooks.py` Shared post-screen SI (cron screener + orchestrator): health, auto-relax on zero candidates `utils/adaptive_rsi_reconciliation.py` Integrity scan `adaptive_rsi_screener_drift` when screener prefilter lags entry relax

Hooks: `orchestrator.py` post-screen and entry paths; **agents/screener_agent.py** end-of-run (intraday cron); `agents/recursive_evolution.py` phase-1 queue process

Data: `data/si_recommendation_queue.json`, `data/entry_si_overrides.json`, `data/screener_si_overrides.json`, `data/last_screening_meta.json` (includes `adaptive_rsi` telemetry)

Reads sibling: `fortress-ai/data/si_recommendation_summary.json`, `data/si_capability/overrides.json`

Adaptive RSI policy (June 2026): The system **always strives for adaptive RSI** - when fill-recency gap persists, entry SI raises relaxed `rsi_cap` (up to 70). The screener prefilter must use the same ceiling via `tier_rsi_threshold()` and `effective_bear_tier1()`; ranging-extremes bands use `adaptive_ranging_oversold_cap()` instead of fixed 35. Misalignment is flagged by integrity scan code `adaptive_rsi_screener_drift`.

Key env vars:

Variable Purpose `FORTRESS_CLASSIC_SI_AUTO` Master Classic SI autonomous cycle
`FORTRESS_CLASSIC_SI_SCREENER` Screener relax auto-apply `FORTRESS_CLASSIC_SI_ENTRY` Entry-gate relax auto-apply `FORTRESS_CLASSIC_ENTRY_FILL_RECENCY_LOOSEN` Enable fill-recency loosen `FORTRESS_CLASSIC_FILL_RECENCY_LOOSEN_DAYS` Days-without-fill threshold

FORTRESS_CLASSIC_FILL_RECENCY_RSI_CAP Relaxed RSI cap before deterministic gate
FORTRESS_CLASSIC_FILL_RECENCY_LLM_MIN_CONF Relaxed LLM min confidence
FORTRESS_MAX_ORDER_NOTIONAL_USD Exit chunk cap (default 25000)

3.4 Agentic overlay (pre-open / planning)

Command Output artifact scout_swarm scout_opportunity_queue_.json analyst_ensemble analyst_consensus_*.json cio_cycle cio_directive_*.json multi_timeframe multi_timeframe_plan_*.json sector_rotation sector_rotation_signal_*.json geographic_allocation geographic_allocation_plan_*.json*

3.5 Fortress hedge book (orchestrator fortress)

Multi-asset sleeve recommendations: commodities, forex, VIX insurance, theta spreads, dividend capture, pairs. Output: *fortress_report_*.json*, *hedging_recommendations.json*.

4. Command Center dashboard - panel reference

URL: <http://HOST:8083/> (COMMAND_CENTER_PORT)

Template: [dashboard/templates/command_center.html](#)

Auth: Optional HTTP Basic (FORTRESS_DASHBOARD_USER / FORTRESS_DASHBOARD_PASS)

Panel 1: Header / operator controls

Purpose: Instance identity, halt/resume, navigation to key panels.

Control API Trading importance Halt trading GET/POST /api/operator/halt Emergency stop - blocks new entries Resume Same Clears operator halt Policy rollback clear POST /api/policy/clear_rollback Exit forced capital_preservation profile

Data: *utils/operator_halt.py*, env FORTRESS_TRADING_HALT=1

Panel 2: Alerts feed

API: */api/alerts_feed*

Purpose: Real-time operator alerts (errors, guardrail trips, stale cron).

Metric Source Why it matters Alert list Aggregated from logs + health checks First line of incident response Severity / timestamp Alert generator Triage priority

Panel 3: Operator morning brief

API: */api/operator_brief*

Purpose: Start-of-day summary for the operator - what ran overnight, what to watch today.

Metric Source Why it matters Brief text / sections Generated brief artifact Reduces manual log reading Staleness indicators Cron heartbeats Missed jobs surface here

Panel 4: TradingView signal queue

API: */api/tradingview_signals*

Purpose: Ingest external TV webhook alerts - **research/queue only**, not auto-trade by default.

Metric Source Why it matters Queued signals data/tradingview_signal_queue.jsonl External alpha ideas Signal metadata Webhook payload Symbol, direction, timestamp

Setup: *docs/TRADINGVIEW_WEBHOOK_SETUP.md*

Panel 5: System health

API: */api/health, /api/system_health*

Purpose: Process, cron, and dependency health.

Metric Source Why it matters Overall status Health aggregator Green/yellow/red ops state Cron heartbeats data/cron_heartbeats.json vs deploy/cron_manifest.json Stale jobs Heartbeat age Screen/monitor may be dead

Panel 6: Performance & trading snapshot

API: /api/performance

Purpose: Primary P&L and win-rate panel - canonical performance view.

Metric Field / computation Source Trading importance Account equity Alpaca get_account() Live broker Capital base Realized P&L (30d) Sum from ledger data/pnl_ledger.jsonl Ground truth closed trades Win rate (30d) wins / closed Same ledger Hit rate Open positions count Alpaca + positions.json Reconciled state Exposure on Classic account Unrealized P&L Alpaca positions Mark-to-market Open risk Per-strategy breakdown Grouped stats Ledger + decisions Which sleeves work Latest screening summary Embedded last_screening_meta.json Funnel context - LATEST SCREENING panel

Why it matters: External reviewers should treat pnl_ledger.jsonl as the authoritative realized performance record.

Panel 7: Safety status

API: /api/safety_status

Purpose: Risk guardian and compliance state - "is it safe to trade?"

Metric Source Trading importance Trading halted Operator halt + env Hard stop Circuit breaker active risk_guardian_state.json Consecutive loss protection Strict mode Policy profile Tighter gates Consecutive losses Risk state Streak counter Auto-reset window Env + state When breaker clears VIX tier caps daily_risk_params.json Regime-scaled limits Execution mode autonomous vs human_in_loop HITL vs auto

Panel 8: Stabilization monitor

Source: Safety + drift panels combined

Purpose: Composite view when system is in degraded/stabilization mode (policy rollback, drift alert).

Metric Source Why it matters Rollback active policy_rollback_state.json Forced capital_preservation Drift alert drift_report.json Performance regression detected

Panel 9: Go-live scorecard

Purpose: Composite readiness gates before promoting shadow -> live.

Gate Typical check Why it matters Win rate threshold 30d ledger Minimum edge Walk-forward pass walk_forward_report.json OOS validation Drift clear No active alert Stability Cron fresh Heartbeats Ops reliability Halt clear Not halted Operator approval

Reference: docs/TRUST_AND_GO_LIVE_PLAYBOOK.md

Panel 10: Bot audit (objectives)

API: /api/bot_audit

Purpose: Objective scoring - is the bot meeting its stated goals?

Metric Source Why it matters Objective scores bot_audit_agent output Structured self-assessment Gap list Audit rules What to fix next

Panel 11: Fortress intelligence brief

API: /api/intelligence_brief

Purpose: Daily post-close self-QA report - scout gaps, unexecuted BUYs, optimization queue.

Metric Source Why it matters Brief summary fortress_intelligence_brief_.md Human-readable daily review Machine JSON fortress_intelligence_brief_*.json Automation input Optimization queue Brief generator*

Generated by: orchestrator.py generate_intelligence_brief

Panel 12: Recursive evolution status

API: /api/evolution_status

Purpose: 5-phase self-improvement cycle output.

Metric Source Why it matters Last cycle timestamp recursive_evolution_.json Freshness Diagnosis summary
Phase 1 What went wrong Param proposals Phase 2 RSI/volume/stop tuning Thompson sampling allocations
Phase 3 Strategy A/B weights Change plan Phase 4 Proposed writes (dry-run default) Meta-learning trend
meta_learning_state.json Bandit state*

Safety: Writes require FORTRESS_EVOLUTION_ALLOW_WRITES=1

Panel 13: Recursive intelligence stack

APIs: /api/critique_stats, /api/regime_status, /api/sentiment_velocity,
/api/options_flow, /api/earnings_intel, /api/cross_asset

Purpose: Live feeds from parallel intelligence agents.

*Feed Data file Update cadence Trading use Regime daily_risk_params.json 30 min RTH VIX-tier risk caps
Sentiment velocity sentiment_velocity.json 20 min RTH Social/news momentum Options flow
options_flow.json 30 min RTH Unusual activity Earnings intel earnings_intel.json 8:00 + 16:30 ET Event risk
Cross-asset cross_asset_signal.json Periodic Macro alignment Critique loop In-memory stats Per signal
CONFIRM/MODIFY/REJECT*

Panel 14: Prompt evolution & reflection

APIs: /api/prompt_evolution, /api/reflection_trend

Purpose: LLM prompt tuning and nightly trade reflection scores.

*Metric Source Why it matters Active prompts prompt_store.json Current LLM instructions Reflection scores
reflection_log.json Quality trend Evolution events Prompt evolution agent Change audit*

Gated by: FORTRESS_PROMPT_EVOLUTION_ENABLED, FORTRESS_REFLECTION_ALLOW_WRITES

Panel 15: Morning briefing

API: /api/morning_briefing

Purpose: Pre-market operator summary from briefing_agent (~8:45 ET).

*Metric Source Why it matters Brief text Briefing artifact Day setup Key levels / events Agent output Context for
screen window*

Panel 16: LLM usage & spend

APIs: /api/llm_usage, /api/llm_credits

Purpose: Token/cost telemetry - "billing proxy" for LLM calls.

*Metric Source Why it matters Calls today / 7d / lifetime api_costs.jsonl Activity volume Tokens in/out Same
Usage detail USD spend utils/cost_calculator Cost control Model breakdown Aggregated Ollama (free) vs
DeepSeek Credits remaining License tier SaaS gating if enabled*

Panel 17: Headline event agent

API: /api/headline_event_status

Purpose: Shadow agent reacting to headline events (sketch/production flag).

Metric Source Why it matters Shadow log tail headline_event_shadow_.jsonl Event-driven ideas Agent status
Runtime state Enabled/shadow mode*

Panel 18: Pristine verification

API: /api/pristine

Purpose: Data integrity checks - positions.json vs Alpaca, ledger consistency.

Metric Source Why it matters Reconciliation status Pristine checker Split-brain detection Mismatch details Diff output Fix before trading

Panel 19: Agent activity

API: /api/agents

Purpose: Which agents ran recently and their outcomes.

Metric Source Why it matters Agent list Cron heartbeats + run registry Coverage map Last run / status operational_runs.jsonl Staleness per agent

Panel 20: Recommendations

API: /api/recommendations

Purpose: Aggregated recommendation blobs from multiple analyzers.

Source file Type opportunity_recommendations.json Bullish setups defensive_recommendations.json Risk-off actions regime_recommendations.json Regime shifts strategic_recommendations.json Longer-term pattern_discovery_recommendations.json LLM-discovered patterns no_trade_findings.json Why not to trade

Panel 21: Prioritized action queue

API: /api/action_queue

Purpose: Operator to-do list synthesized from briefs, drift, and audit.

Metric Source Why it matters Queue items Action synthesizer What to do next Priority / category Scoring rules Triage order

Panel 22: Operator run registry

API: /api/operator_runs

Purpose: Audit trail of orchestrator invocations.

Metric Source Why it matters Run ID / command / timestamp operational_runs.jsonl Who ran what when Exit code / duration Run metadata Success/failure

Panel 23: Trust ledger

API: /api/trust_ledger

Purpose: Governance events - halts, rollbacks, manual overrides.

Metric Source Why it matters Event type / actor / timestamp trust_ledger.jsonl Compliance audit Reason text Logged context Why action was taken

Panel 24: Why this trade?

API: /api/why_trade

Purpose: Explain a specific trade decision - screener path, gates passed/failed.

Metric Source Why it matters Decision chain decisions_log.jsonl Full gate trace Skip reasons Screening funnel Transparency

Panel 25: One-click runbooks

API: /api/runbooks

Purpose: Operator shortcuts to safe diagnostic commands.

*Runbook Action Why it matters Health check check_health.py Quick sanity Risk diagnostic
diagnose_risk_guardian.py Breaker state LLM check check_llm_provider.py Provider connectivity*

Reference: docs/OPERATOR_RUNBOOKS.md

Panel 26: Drift status

API: /api/drift

Purpose: Performance regression detection vs baseline.

*Metric Source Why it matters Drift alert active drift_report.json Triggers policy rollback Metric deltas Drift
detector What regressed Baseline window Config Comparison period*

Panel 27: Trust report (aggregate)

API: /api/trust_report

Purpose: Composite trust score across gates, drift, pristine, cron.

*Metric Source Why it matters Composite score Aggregator Single go/no-go indicator Component breakdown
Sub-checks Where trust is weak*

Panel 28: Price chart (daily)

API: /api/chart_bars

Purpose: Daily OHLC chart for watchlist symbol.

*Metric Source Why it matters Bar series yfinance / Alpaca Visual context Symbol selector Watchlist
Multi-ticker view*

Panel 29: Hedging - commodities & forex

Source: Performance + fortress artifacts

Purpose: Multi-asset hedge book recommendations from fortress_orchestrator.

Metric Source Why it matters Regime label fortress_report_.json Macro posture Commodity recs Hedge
sleeves Diversification Forex hedge forex_hedger Currency risk Theta / dividend / pairs Sleeve agents Income
+ relative value*

Panel 30: News impacting strategy

API: /api/news

Purpose: News items that may affect open positions or today's screen.

*Metric Source Why it matters Headlines yfinance / news APIs Event risk Symbol tags Enrichment
Position-specific alerts*

Panel 31: Open positions

API: /api/positions

Purpose: Live Alpaca positions with P&L.

*Column Source Trading importance Symbol / Qty / Side Alpaca Exposure Avg entry / Current Alpaca Cost
basis Unrealized P&L Alpaca Open risk Market value Alpaca Notional*

Panel 32: Recent orders

API: /api/orders

Purpose: Recent Alpaca order history.

Column Source Why it matters Order ID / status Alpaca Fill confirmation Symbol / qty / side Alpaca Audit Submitted / filled time Alpaca Latency

Panel 33: Live activity feed

API: /api/feed

Purpose: Streaming-style log of recent system events.

Metric Source Why it matters Event stream Logs + decisions tail Real-time ops view Event type Classifier
Filter by category

5. Self-improvement & learning (multi-path)

Classic Fortress has **layered** improvement loops - deterministic-first, LLM-assisted, all gated by env flags.

System Command / module Learns Output Gate SI recommendation queue
utils/si_recommendation_queue.py Integrity + capability findings data/si_recommendation_queue.json
FORTRESS_CLASSIC_SI_AUTO Classic SI autonomous utils/classic_si_autonomous.py Heuristic assess +
auto-apply queue dispositions default ON Fill-recency entry utils/fill_recency_entry.py Days since last fill/entry
(positions.json-aware) entry_si_overrides.json FORTRESS_CLASSIC_ENTRY_FILL_RECENCY_LOOSEN
Adaptive RSI (unified) utils/adaptive_rsi.py Screener + entry ceiling alignment last_screening_meta.json ->
adaptive_rsi_fill-recency + entry SI Screener SI utils/classic_si_screener.py Zero-candidate streaks
screener_si_overrides.json FORTRESS_CLASSIC_SI_SCREENER Screener cron hooks
utils/classic_screening_hooks.py Intraday zero-candidate auto-relax pipeline health JSON every 30 min
screener Chunked exits utils/order_sizer.py N/A (execution guard) chunked_exit log markers
FORTRESS_MAX_ORDER_NOTIONAL_USD Param auto-tune orchestrator tune RSI, drop, volume
thresholds current_params.json Manual or evolve Recursive evolution orchestrator evolve 5-phase diagnosis
+ Thompson sampling recursive_evolution_*.json FORTRESS_EVOLUTION_ALLOW_WRITES=1 LLM
learning llm_learning_agent Post-exit lessons llm_lessons.jsonl FORTRESS_LLM_LEARNING_ON_EXIT=1
Intelligence brief generate_intelligence_brief Daily self-QA fortress_intelligence_brief_*. Scheduled
Meta-architect orchestrator architect New agents from loss patterns agents/generated/* 100+ trades required
Prompt evolution prompt_evolution_agent Prompt text prompt_store.json
FORTRESS_PROMPT_EVOLUTION_ENABLED Reflection reflection_agent Nightly trade scores
reflection_log.json FORTRESS_REFLECTION_ALLOW_WRITES Critique loop critique_loop
CONFIRM/MODIFY/REJECT In-flight stats FORTRESS_CRITIQUE_LOOP_ENABLED Walk-forward
walk_forward_validator OOS validation walk_forward_report.json Research gate Drift rollback drift_detector
Auto profile rollback policy_rollback_state.json Automatic on alert Ops autofix ops_autofix_agent Reconcile
stale state ops_autofix_report_*.json Every 15 min

Verify: python3 orchestrator.py verify_learning

Pre-deploy: ./scripts/e2e_before_deploy.sh (required); optional RUN_UNIT_TESTS=1 for full suite.
Integrity scan runs on evolve cron and surfaces adaptive_rsi_screener_drift when screener/entry RSI
diverge.

6. Risk & guardrails

Control Mechanism Env / file Max order notional Exit chunking via order_sizer
FORTRESS_MAX_ORDER_NOTIONAL_USD Fill-recency loosen Bounded entry relax when no recent fill
FORTRESS_CLASSIC_FILL_RECENCY_* Adaptive RSI alignment Screener tier RSI >= entry relaxed cap
when SI active utils/adaptive_rsi.py, integrity adaptive_rsi_screener_drift Classic SI master Autonomous
queue processing FORTRESS_CLASSIC_SI_AUTO Pre-trade gate utils/pre_trade_gate.py All paths Risk
guardian Circuit breaker, streaks risk_guardian_state.json Operator halt Manual + env
FORTRESS_TRADING_HALT=1 Policy profiles opportunistic / capital_preservation policy_profiles.json
Drawdown guard Peak-to-trough limit FORTRESS_MAX_DRAWDOWN_FROM_PEAK Daily loss cap Session
stop FORTRESS_MAX_DAILY_LOSS Sector concentration Correlation guard
FORTRESS_MAX_SECTOR_CONCENTRATION_PCT Velocity guard PnL change per hour
FORTRESS_MAX_PNL_CHANGE_PER_HOUR Position aging Max hold days
FORTRESS_MAX_HOLD_DAYS * Spread filter Data quality FORTRESS_MAX_SPREAD_PCT Entry
blackouts Time windows FORTRESS_ENTRY_BLACKOUT_WINDOWS_ET Live trading ack Explicit opt-in
FORTRESS_LIVE_TRADING_ACK HITL mode Human approval queue
FORTRESS_EXECUTION_MODE=human_in_loop

Reference: docs/guardrail_ticket_stack.md (10 guardrails, 3 phases)

7. Data architecture

Path Purpose data/pnl_ledger.jsonl Canonical realized P&L data/si_recommendation_queue.json Classic SI queue (sibling schema) data/si_recommendation_summary.json Queue + classic_si cycle rollup data/entry_si_overrides.json Fill-recency entry relax state data/screener_si_overrides.json Screener SI relax state data/last_screening_meta.json Screener telemetry + adaptive_rsi block data/screening_pipeline_health.json Consecutive zero-candidate runs (cron + daily screen) data/integrity_scan_latest.json Integrity findings incl. adaptive_rsi_screener_drift data/decisions_log.jsonl All trading decisions + outcomes data/positions.json Open positions (reconcile with Alpaca) data/current_params.json Tunable RSI/drop/volume/stop params data/risk_guardian_state.json Circuit breaker state data/daily_risk_params.json Regime-driven risk data/daily_signals_YYYYMMDD.json Screening funnel data/trust_ledger.jsonl Governance audit data/operational_runs.jsonl Run registry data/api_costs.jsonl LLM token/cost telemetry data/recursive_evolution_*.json Evolution cycles data/llm_lessons.jsonl Extracted LLM lessons data/drift_report.json Performance drift data/cron_heartbeats.json Job health

8. Relationship to Fortress AI

Use case Classic (this repo) Fortress AI Production Oracle bot Yes Research / A-B Intraday 1m skim (16 symbols) No (fortress-ai skim service) Yes LLM every 5 min Optional advisory Core loop (Unified AI) Comparison scripts/compare_systems.py with CLASSIC_DATA_DIR Comparison drawer + classic_bridge.py (read-only; separate Alpaca accounts) Alpaca account Classic paper account (own API keys) Fortress AI paper account (separate keys) Shared halt Optional FORTRESS_SHARED_HALT_PATH Same file pattern (optional cross-stack ops only) Chunked exits utils/order_sizer.py unified_ai/order_executor.py + utils/order_chunking.py SI queue data/si_recommendation_queue.json Sibling queue + autonomous code SI

Classic code is **not modified** when fortress-ai runs side-by-side on the same host. The two stacks use **separate Alpaca accounts and independent capital**; classic_bridge.py reads Classic files and Classic credentials for comparison only.

9. Known limitations

1. Entry window concentrated 14:30-16:00 ET - not full-session intraday for equity screen path.
 2. LLM is advisory by default - rules execute; LLM does not directly submit orders unless configured.
 3. Meta-architect requires 100+ trades before generating new agents.
 4. Evolution writes are opt-in (FORTRESS_EVOLUTION_ALLOW_WRITES=1).
 5. TradingView signals are queued, not auto-executed.
 6. Paper trading default - live requires explicit ack.
 7. Separate Alpaca account from fortress-ai - Classic positions and capital are independent of Fortress AI skim/infra/unified books.
 8. No SI queue dashboard panel - queue is backend-only; review via data/si_recommendation_summary.json or fortress-ai SI APIs.
 9. E2E (./scripts/e2e_before_deploy.sh) validates imports and mocked smoke paths - it does **not** live-test Alpaca fill latency or screener/entry RSI alignment; use integrity scan adaptive_rsi_screener_drift and ./scripts/e2e_before_deploy.sh before every deploy.
-

10. Glossary

Term Definition Classic Fortress This trading-bot multi-agent stack Command Center Flask dashboard on port 8083 Screen Daily candidate funnel + optional entry execution HITL Human-in-the-loop - approved trades in pending queue Policy profile Risk/screening knob set (opportunistic, capital_preservation) Trust ledger Governance audit log Evolution 5-phase recursive self-improvement cycle Classic SI Integrity-driven queue + fill-recency / screener auto-relax adaptive_rsi Unified ceiling (up to 70) shared by screener prefilter and entry gate adaptive_rsi_screener_drift Integrity finding when screener rejects on tighter RSI than entry SI chunked_exit Sell split into child orders under notional cap CIO directive Top-level agentic allocation artifact Sleeve Day/swing/position timeframe bucket

Classic Fortress (KHF Zero-Cost Trading Bot) - Research dashboard - Not investment advice